

COL334-A4

G. Anudeep Pola Venkata Revanth
Entry No: 2022CS51617 Entry No: 2022CS51650

October 30, 2024

Contents

1	Part 1: Reliability and Fast Recovery	2
1.1	Implementation	2
1.1.1	Client-Side :-	2
1.1.2	Server-Side :-	3
1.2	Assumptions/Observations :	3
1.3	Results and Analysis	4
2	Part 2: Congestion Control with TCP Reno	5
2.1	Congestion Control Mechanisms	5
2.2	Implementation	5
2.2.1	Client-Side :-	5
2.2.2	Server-Side :-	6
2.3	Results and Analysis	6
3	Part3 : TCP Cubic	8

Introduction

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are two fundamental transport layer protocols in computer networks. While TCP provides reliable, ordered, and error-checked delivery of data and UDP doesn't guarantee reliability and order.

- **Part 1: Reliability** - Implementation of UDP with reliability features including acknowledgments, retransmissions, fast recovery, packet numbering, and timeout mechanisms.
- **Part 2: Congestion Control** - Implementation of TCP Reno-like congestion control mechanisms including slow-start, congestion avoidance, and fast recovery.
- **Part 3** - Implementation of TCP CUBIC congestion control algorithm for improved performance.

The goal is to create a client-server application that enables reliable file transfer using UDP.

1 Part 1: Reliability and Fast Recovery

1.1 Implementation

1.1.1 Client-Side :-

Client maintains a buffer to store out-of-order packets so that server won't need to retransmit it later.

- Client sends 'START' message until it receives some data
- Pseudo code for packet handling

```
def receive_file():
    .....
    if expected_seq_num == received_seq_num:
        # expected_seq_num += len(recv_data)
        # write data to file
        while (there is packet with seq_num == expected_seq_num in buffer):
            # take that packet from buffer and write to file
            # expected_seq_num += len(packet_taken_from_buffer)
            send_ack_packet(expected_seq_num) #cumulative ack
    elif received_seq_num < expected_seq_num:
        send_ack_packet(expected_seq_num)
    else:
        # insert the packet in buffer
```

```
send_ack_packet(expected_seq_num)
```

```
.....
```

- After receiving the end packet client sends an ack saying that it received end packet and ends it's connection with server

1.1.2 Server-Side :-

- Server starts sending data after it receives 'START' message
- After sending packets in the window server starts the timer which will depend on the **rto**(retransmission time out) and if it didn't receive any correct ack packet (fast recovery case is explained below) we retransmit all packets in the window
- Pseduocode for ack handling

```
def send_file:
.....
    try:
        if last_ack_num > recv_ack_num:
            # remove all packets from window whose seq_num is less than recv_ack_num
            # last_ack_num=recv_ack_num
            # calculate rtt and update rto value,window_size(if used*)
        elif last_ack_num < recv_ack_num:
            # continue
        else:
            #duplicate_ack_cnt+=1
            if(duplicate_ack_cnt > duplicate_threshold):
                fast_recovery(#send first packet in window)
    except:
        retransmit_all_pkts_in_window()
        rto=2*rto
        # set timer
.....
```

- Server stops its connection after it receives end ack pkt from the client or else it will continue to send the EOF again

1.2 Assumptions/Observations :

Regarding Window Size

- We have implemented a variable window to maintain a speed of 50 Mb/s. However, due to random fluctuations in RTT, the window size often becomes too large. When RTT reduces again and the RTO is small compared to the number of packets in the window, all packets are retransmitted. This leads to high redundancy and bandwidth wastage.

- We can update the window size using the following formula:

$$\text{window_size} = (\text{rtt} * 50 * 1e6) // (\text{MSS} * 8)$$

1.3 Results and Analysis

Impact of Packet Loss on Transmission Time

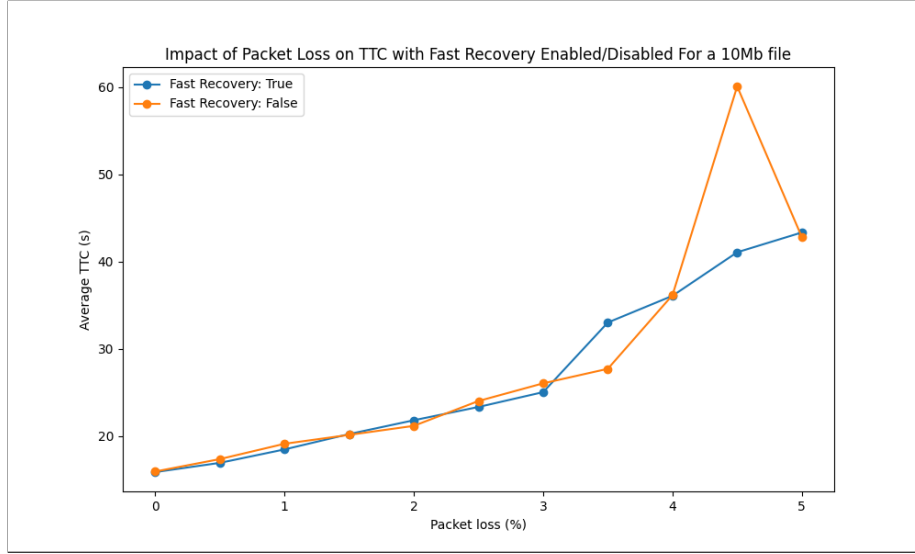


Figure 1: TTC vs. Packet Loss with and without Fast Recovery

The above graph shows that generally, Fast Recovery is more effective. Figure 1 shows how TTC change upon using fast recovery. Fast recovery generally reduces the transmission time by retransmitting lost packets quickly upon detecting three duplicate acknowledgments without waiting for time out(early detection).

Our implementation goes out of fast recovery mode immediately after receiving a new acknowledgement. But in the case of multiple packet losses the new acknowledgement doesn't necessarily acknowledge all the lost packets (lost before entering fast recovery). So immediately after exiting fast recovery it finds another lost packet and enters fast recovery with a fresh timeout. This leads to a series of entering fast recoveries. Instead if we had just waited for timeout amount without fast retransmitting we might have recovered more lost packets in a shorter time than fast recovery. So TCP without fast retransmit tends to perform better than that with fast retransmit in case of larger loss rates.

Impact of Delay on Transmission Time

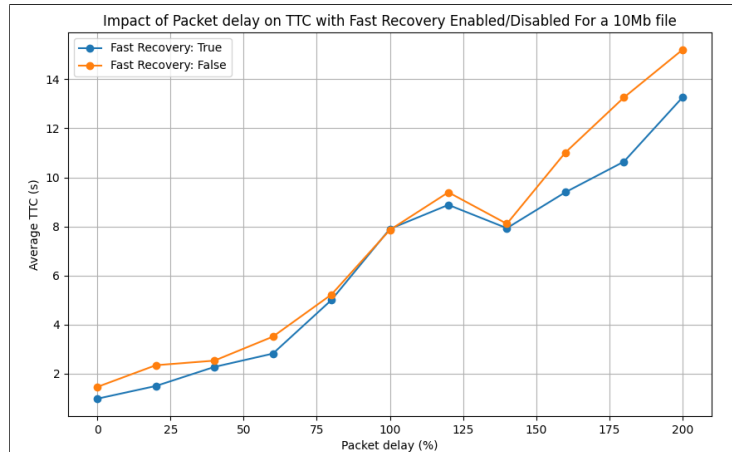


Figure 2: Transmission Time vs. Link Delay with and without Fast Recovery

From the graph we can see that as delay increases TTC increases and its value is lower when we use fast recovery with it.

2 Part 2: Congestion Control with TCP Reno

2.1 Congestion Control Mechanisms

Design Overview

The congestion control implementation follows TCP Reno principles:

- Slow start with exponential window growth
- Congestion avoidance with linear window growth
- Fast recovery mechanism
- Timeout handling with window reset

2.2 Implementation

2.2.1 Client-Side :-

- Client implementation remains same as [part-1 client side](#)

2.2.2 Server-Side :-

Here the modified was the congestion window (cwnd) is dynamically adjusted based on the acknowledgments (acks) received and packet loss events. The cwnd increases with each ack received, allowing for controlled growth. When packet loss is detected, the cwnd is reduced to half of its current size, and fast recovery is implemented similarly like above.

2.3 Results and Analysis

Throughput vs. Link Delay

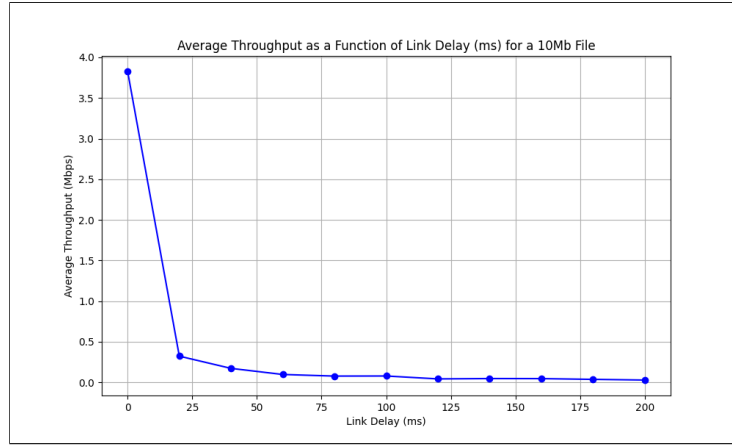


Figure 3: Average Throughput vs. Link Delay

As shown in Figure 3, throughput decreases as delay increases, in line with the relation $\text{Throughput} \propto \frac{1}{RTT \times \sqrt{p}}$. This result aligns with existing literature on TCP Reno performance under increased latency.

Throughput vs. Packet Loss

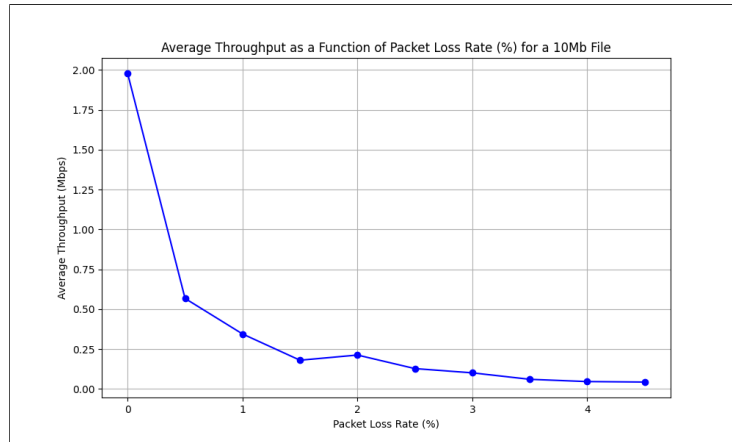


Figure 4: Average Throughput vs. Packet Loss

Figure 4 demonstrates that as packet loss increases, throughput decreases sharply. This behavior highlights the sensitivity of TCP Reno's congestion control to packet loss, due to reliance on duplicated ACKs and fast recovery for minor congestion events.

Fairness Analysis

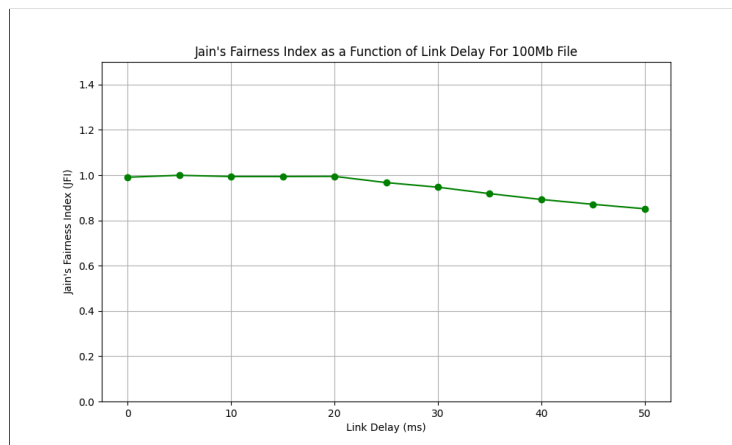


Figure 5: Jain's Fairness Index as a Function of Link Latency

Figure 5 illustrates Jain's Fairness Index across varying link latencies. A higher index at moderate latencies indicates fairer bandwidth allocation, as congestion

control stabilizes the window size based on observed network conditions.

3 Part3 : TCP Cubic

It uses a cubic function to adjust the CWND. This function allows for a more aggressive increase in the window size, especially after recovering from congestion. The cubic function is defined as:

$$W(t) = C(t - K)^3 + W_{\max}.$$

In TCP CUBIC, after a congestion event, the CWND initially grows slowly, then accelerates as it moves away from the congestion point, and finally slows down as it approaches $W_{\max}$. This behavior allows TCP CUBIC to utilize available bandwidth more efficiently and recover faster from congestion compared to TCP Reno.

Throughput Analysis for Long Delay Path

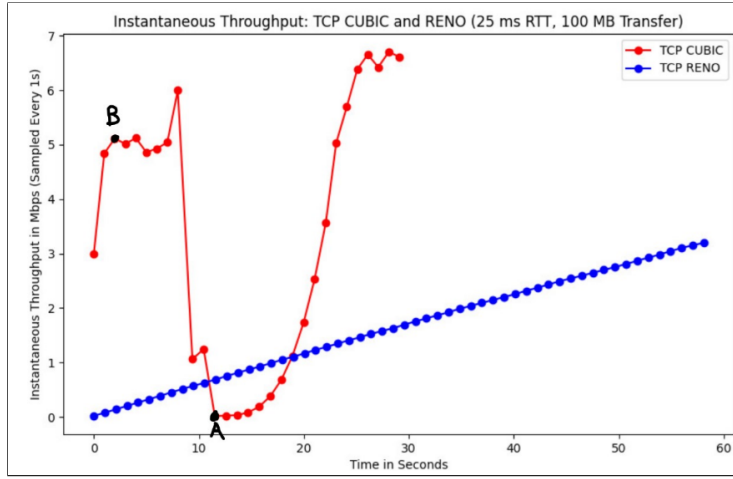


Figure 6: Throughput Comparison for TCP CUBIC and TCP Reno (25 ms Delay)

Points A and B

Point A shows a moment where TCP CUBIC's throughput dips briefly due to congestion. This event resets TCP CUBIC's congestion window (CWND), causing it to start growing again based on the CUBIC algorithm. After this dip, the CWND increases but slows down as it gets close to the maximum limit, $W_{\max}$. This is visible in the flatter part of the curve, indicating that TCP CUBIC adjusts its growth to avoid large increases near the maximum limit.

TCP Reno Behavior

On the other hand, TCP Reno's throughput (represented by the blue line) steadily increases without major drops. This stability comes from its AIMD (Additive Increase Multiplicative Decrease) algorithm, which allows for slow and controlled growth. Unlike CUBIC, TCP Reno avoids sharp fluctuations by not pushing the congestion window as close to its maximum, leading to fewer sudden drops in throughput.

Throughput Analysis for Short Delay Path

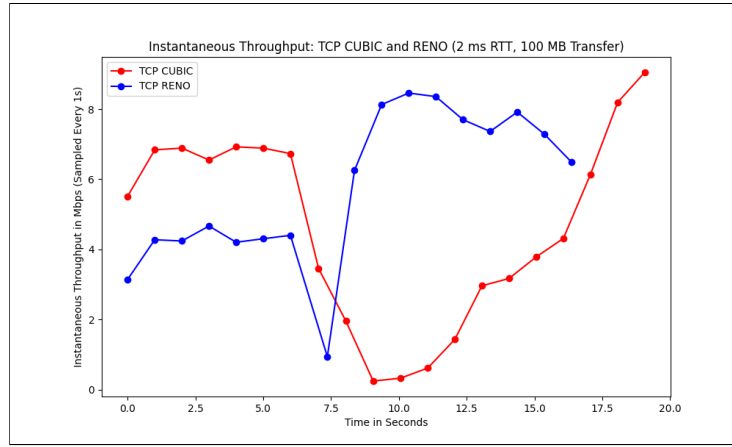


Figure 7: Throughput Comparison for TCP CUBIC and TCP Reno (2 ms Delay)

In the throughput analysis for the short delay path (2 ms), TCP CUBIC showed a small drop in throughput, marked by the lowest point in the red curve. This point represents TCP CUBIC's inflection, where it gets close to its maximum congestion window ($W_{\max}$). On the other hand, TCP Reno kept its throughput steady and finished its execution earlier than TCP CUBIC. This indicates that TCP Reno's stable growth helps it perform better in low-latency situations.